SAWNERGY: Refined End-to-End Research Pipeline

1. Expand SAWNERGY's MD Feature Extraction

Extend SAWNERGY so that it can extract a richer set of raw, residue-level features from molecular dynamics trajectories.

Potential features include:

- Residue-residue distances
- van der Waals interaction energies
- Electrostatic interaction energies
- Hydrogen bonds
- Bonded interactions
- Bond lengths
- Bond angles
- Dihedral/torsional angles
- Contact maps
- Residue center-of-mass positions
- Protein-DNA interaction features
- Protein-ligand interaction features
- Other physically meaningful residue-wise quantities available through CPPTRAJ or related tools

The objective is to transform every MD frame into a structured representation containing substantially more information than coordinates alone.

---

2. Define the Frame Representation

Represent each simulation frame as a collection of residue-residue interaction matrices.

A natural representation is a multi-channel tensor:

$$X_t \in \mathbb{R}^{N \times N \times C},$$

where:

- N is the number of residues/nodes,
- C is the number of interaction or geometric feature channels.

Example channels could include:

$$X_t = \{D_t,\, E_t^{vdW},\, E_t^{elec},\, H_t,\, C_t,\, A_t, \ldots\}.$$

Possible representation strategies:

Option A — Multi-channel matrices

Treat each physical quantity as a separate $N \times N$ channel.

This naturally supports CNN-style encoders.

Option B — Dynamic graphs

Treat residues as nodes and residue-residue quantities as edge features.

Each frame becomes:

$$G_t = (V, E_t, X_t, E_t^{features}).$$

This naturally supports GNN encoders and may better reflect the underlying molecular structure.

Option C — Hybrid representation

Use graph structure for residue connectivity while separately retaining dense interaction matrices when useful.

This is likely the most expressive approach but also the most complex.

---

3. Pretrain a Frame-Level Autoencoder

Before modeling entire trajectories, train an autoencoder that learns how to represent an individual MD frame.

The model has the form:

$$X_t \xrightarrow{E_f} h_t \xrightarrow{D_f} \hat{X}_t,$$

where:

- X_t is the raw frame representation,
- h_t is the learned frame embedding,
- $\hat{X}_t$ is the reconstructed frame.

Possible frame encoders:

Option A — CNN Autoencoder

Appropriate if frames are represented primarily as dense multi-channel residue-residue matrices.

Option B — Graph Autoencoder

Appropriate if frames are represented as residue interaction graphs.

Option C — Graph/CNN hybrid

Use graph operations for topology-sensitive information and convolutions for dense pairwise matrices.

The purpose of this stage is to reduce each high-dimensional frame to:

$$h_t \in \mathbb{R}^{d_f},$$

where h_t preserves the physically meaningful state of the system at time t.

---

4. Build a Trajectory-Level Sequence Autoencoder

Given a trajectory:

$$X_1, X_2, \ldots, X_T,$$

first encode each frame:

$$h_t = E_f(X_t).$$

The resulting sequence

$$h_1, h_2, \ldots, h_T$$

is passed to a temporal encoder:

$$(h_1, \ldots, h_T) \xrightarrow{E_{\text{traj}}} z_{\text{traj}},$$

where

$$z_{\text{traj}} \in \mathbb{R}^{d_z}$$

is a single embedding representing the complete MD trajectory.

Possible temporal encoders:

Option A — LSTM

A natural first implementation for recurrent sequence compression.

Option B — GRU

Similar to the LSTM but simpler and potentially easier to train.

Option C — Transformer / temporal attention model

Potentially more appropriate for very long trajectories because it avoids compressing all temporal information through recurrent state transitions.

Option D — Hierarchical temporal encoder

Encode short trajectory windows first and subsequently encode the resulting window-level embeddings into one trajectory-level vector.

This may be preferable for extremely long MD trajectories.

---

5. Reconstruct the Entire Trajectory From the Trajectory Embedding

The trajectory decoder should reconstruct the sequence of frame embeddings:

$$z_{\text{traj}} \to \hat{h}_1, \hat{h}_2, \ldots, \hat{h}_T.$$

The pretrained frame decoder then reconstructs the physical frames:

$$\hat{h}_t \xrightarrow{D_f} \hat{X}_t.$$

Therefore:

$$(X_1, \ldots, X_T) \to z_{\text{traj}} \to (\hat{X}_1, \ldots, \hat{X}_T).$$

After frame-level pretraining, jointly fine-tune:

- the frame encoder,
- trajectory encoder,
- trajectory decoder,
- frame decoder.

This allows frame representations to adapt to information that is important at the trajectory level.

---

6. Use a DNA-Weighted Reconstruction Objective

Do **not** restrict the representation exclusively to protein-DNA interactions.

The model should observe the entire molecular system because the biological effect may propagate through an allosteric pathway such as:

$$\begin{aligned}\text{ligand binding} &\to \text{local structural changes} \to \text{protein dynamics} \\ &\to \text{DNA-binding domain} \to \text{protein-DNA interactions}.\end{aligned}$$

Instead, reconstruct the entire system while assigning greater importance to errors involving DNA interactions.

For example:

$$\mathcal{L}_{\text{recon}} = \lambda_{\text{global}} \mathcal{L}_{\text{global}} + \lambda_{\text{DNA}} \mathcal{L}_{\text{DNA}},$$

with

$$\lambda_{\text{DNA}} > \lambda_{\text{global}}.$$

Possible experiments:

Option A — Whole-system reconstruction

Use uniform reconstruction loss everywhere.

This provides a baseline.

Option B — DNA-only representation

Train using only protein-DNA interactions.

This tests whether global protein dynamics actually contribute useful information.

Option C — Whole-system representation with DNA-weighted loss

Use the entire system but penalize protein-DNA reconstruction errors more heavily.

This is the preferred initial formulation.

---

7. Ensure the Trajectory Latent Space Represents Biological Similarity

Reconstruction alone does not guarantee that Euclidean or cosine distance in trajectory latent space corresponds to biological similarity.

Initially, train the model purely with reconstruction and test whether meaningful dynamical states emerge naturally.

If necessary, introduce an additional metric-learning objective:

$$\mathcal{L} = \mathcal{L}_{\text{reconstruction}} + \lambda_{\text{metric}} \mathcal{L}_{\text{metric}}.$$

Possible approaches:

Option A — Pure unsupervised reconstruction

Allow latent geometry to emerge naturally.

This should be tested first.

Option B — Contrastive learning

Encourage trajectories representing biologically similar systems to have similar embeddings.

For example:

$$z_{\text{WT}_1} \approx z_{\text{WT}_2}.$$

Option C — Triplet loss

Construct triplets such as:

$$(\text{WT}_1, \text{WT}_2, \text{mutant})$$

and explicitly enforce:

$$d(z_{\text{WT}_1}, z_{\text{WT}_2}) < d(z_{\text{WT}_1}, z_{\text{mutant}}).$$

Option D — Supervised or semi-supervised metric learning

Use available biological labels, mutation identities, functional measurements, or known rescue behavior when available.

---

8. Train the Trajectory Encoder on a Large MD Corpus

Train the trajectory representation model on as many suitable MD simulations as possible.

The training corpus could include:

- WT p53 trajectories
- Y220C trajectories
- Other p53 mutants
- Ligand-bound systems
- Apo systems
- Different conformational states

- Potentially external p53 MD datasets
- Potentially broader protein MD datasets during representation pretraining

Possible training strategies:

Option A — p53-only training

Produces a highly specialized representation.

Option B — Broad protein pretraining followed by p53 fine-tuning

Could allow the model to learn general principles of protein dynamics before specializing.

Option C — Self-supervised pretraining on large MD datasets

Use reconstruction, masked-frame prediction, temporal prediction, or related self-supervised objectives before training the final trajectory autoencoder.

---

9. Construct a Wild-Type Dynamical Reference

Do not represent wild-type dynamics using one simulation alone.

Run or collect multiple independent WT trajectories:

$$z_1^{WT}, z_2^{WT}, \ldots, z_n^{WT}.$$

These embeddings define a **WT dynamical distribution or manifold**.

Possible WT reference representations:

Option A — Mean WT embedding

$$\mu_{WT} = \frac{1}{n}\sum_{i=1}^{n} z_i^{WT}.$$

Then use:

$$D_{WT}(z) = \|z - \mu_{WT}\|.$$

Option B — Cosine similarity to the WT centroid

Useful if vector direction is more meaningful than magnitude.

Option C — Mahalanobis distance

Estimate WT covariance:

$$\Sigma_{WT}$$

and compute:

$$D_{WT}(z) = \sqrt{(z - \mu_{WT})^T \Sigma_{WT}^{-1} (z - \mu_{WT})}.$$

This accounts for dimensions in which natural WT trajectories vary substantially.

Option D — Learned WT density

Fit a model such as:

- Gaussian mixture model,
- normalizing flow,
- kernel density estimator,
- one-class model

to the distribution of WT trajectory embeddings.

This allows the model to represent a potentially multimodal WT dynamical landscape.

---

10. Establish the Untreated Y220C Baseline

Generate multiple untreated Y220C trajectories and embed them:

$$z_1^{Y220C}, \ldots, z_m^{Y220C}.$$

Measure their distance from the WT distribution.

This establishes the baseline dynamical deviation caused by Y220C.

The model should demonstrate that, statistically:

$$D_{WT}(z^{Y220C}) > D_{WT}(z^{WT})$$

before the representation is used as a rescue metric.

This is a critical validation step.

---

11. Curate an Initial Y220C-Binding Molecule Set

Construct an initial compound dataset containing molecules known or predicted to bind to the Y220C pocket.

Possible sources include:

- experimentally validated Y220C binders,
- known p53 stabilizers,
- literature compounds,
- docking-screen hits,
- chemically diverse analogues,
- intentionally weak or inactive compounds as negative controls.

For every compound m_i, construct and simulate:

$$\mathrm{Y220C} + m_i.$$

Where computationally feasible, perform multiple independent MD replicates per compound.

---

12. Compute an MD-Based Dynamical Rescue Score

For every compound:

$$m_i \to \mathrm{MD} \to X^{(i)} \to E_{\mathrm{traj}} \to z_i.$$

Measure how close the compound-treated Y220C trajectory is to the WT dynamical distribution.

A simple score could be:

$$S_i = -D_{WT}(z_i).$$

A more interpretable normalized rescue score is:

$$R_i = 1 - \frac{D_{WT}(z_i)}{D_{WT}(z_{\mathrm{Y220C}})}.$$

Interpretation:

- $R_i \approx 0$: little or no rescue
- $R_i > 0$: dynamics moved toward WT
- $R_i \to 1$: strong restoration toward WT
- $R_i < 0$: compound moved dynamics farther away from WT

If multiple Y220C reference simulations exist, use their mean or expected WT distance rather than a single baseline trajectory.

---

13. Validate That the Rescue Score Is Biologically Meaningful

Before using the score for molecular optimization, test whether it corresponds to known biology.

Possible validation criteria include:

- Known rescuing compounds should score better than inactive molecules.
- Strong binders should not necessarily score highly unless they actually restore WT-like dynamics.
- WT trajectories should score as WT-like.
- Untreated Y220C should consistently score poorly.
- Known functional measurements should correlate with the learned score when available.
- DNA-binding behavior should improve as the score improves.
- Relevant structural and interaction features should move toward WT values.

This is arguably the most important scientific validation in the entire pipeline.

---

14. Train a Molecular Surrogate Scoring Model

The MD simulations produce a dataset:

$$\mathcal{D} = \{(m_i, R_i)\}_{i=1}^{N}.$$

Train a neural network:

$$f_\theta(m_i) \approx R_i.$$

The purpose of this model is to approximate the expensive MD-based rescue score without actually running an MD simulation.

Possible molecular representations:

Option A — Molecular graph + GNN

Represent:

- atoms as nodes,
- bonds as edges,

and use a molecular GNN.

This is the preferred initial approach.

Option B — SMILES transformer

Represent molecules directly from their SMILES strings using a pretrained chemical language model.

Option C — Molecular fingerprints

Use fingerprints such as Morgan fingerprints as input to an MLP or another regression model.

This provides a simpler baseline.

Option D — 3D molecular representation

Use a geometric GNN operating on 3D conformations.

Potentially more physically informative but substantially more complicated.

---

15. Predict Both Rescue Score and Model Uncertainty

The surrogate should ideally estimate:

$$\hat{R}(m)$$

and uncertainty:

$$U(m).$$

Possible uncertainty approaches include:

- Deep ensembles
- Monte Carlo dropout
- Bayesian neural networks
- Evidential regression
- Ensemble variance

Uncertainty is important because the surrogate will eventually evaluate molecules outside its original training distribution.

---

16. Train or Fine-Tune a Molecular Diffusion Model

Use a molecular generative model capable of proposing new candidate compounds.

Possible approaches:

Option A — Train a diffusion model from scratch

Likely requires a very large molecular dataset and substantial compute.

Option B — Start from a pretrained molecular diffusion model

Fine-tune or condition it for molecules compatible with the Y220C pocket.

This is likely much more practical.

Option C — Use another molecular generative model

The overall SAWNERGY pipeline does not fundamentally depend on diffusion.

Other possible generators include:

- autoregressive graph generators,
- SMILES language models,
- VAEs,
- flow models.

Diffusion is one attractive option rather than a strict requirement.

---

17. Guide Molecular Generation Using the Surrogate Rescue Model

Use:

$$f_\theta(m)$$

as guidance toward molecules predicted to produce strong WT-like rescue.

The generative model should not optimize only the predicted rescue score.

Other constraints should also be considered, such as:

- Y220C pocket compatibility
- Chemical validity
- Synthetic accessibility
- Molecular stability
- Drug-likeness
- Molecular diversity
- Toxicity or undesirable chemical motifs
- Binding affinity estimates
- ADMET-related properties where appropriate

The optimization target therefore becomes multi-objective rather than simply:

$$\max_m \hat{R}(m).$$

18. Select Molecules Using an Active-Learning Acquisition Strategy

Do not simply select the molecules with the highest predicted scores.

Doing so risks allowing the generator to exploit errors in the surrogate model.

Instead define an acquisition function combining:

$$A(m) = \alpha \hat{R}(m) + \beta U(m) + \gamma D(m),$$

where:

- $\hat{R}(m)$ = predicted rescue
- $U(m)$ = uncertainty
- $D(m)$ = chemical diversity or novelty

Possible selection strategies:

Option A — Exploitation

Choose molecules with the highest predicted rescue score.

Option B — Uncertainty sampling

Choose molecules for which the scoring model is least certain.

Option C — Diversity sampling

Choose chemically diverse molecules to broaden the training distribution.

Option D — Combined acquisition

Select a mixture of:

- high-scoring compounds,
- uncertain compounds,
- chemically diverse compounds.

This is the preferred strategy.

19. Run MD Simulations on Selected Generated Molecules

For every selected generated molecule:

$$m_j \rightarrow \mathrm{Y220C} + \text{molecule MD} \rightarrow z_j \rightarrow R_j.$$

The trajectory encoder therefore acts as the expensive physics-based oracle for the molecular optimization system.

Whenever computationally feasible, use replicate MD simulations so that the rescue score incorporates simulation variability rather than relying on one stochastic trajectory.

20. Expand the Molecular Rescue Dataset

Add the newly evaluated compounds:

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(m_j, R_j)\}.$$

The dataset therefore becomes progressively larger and more informative.

Importantly, the dataset should contain not only successful molecules but also:

- failures,
- neutral compounds,
- surprising compounds,
- uncertain compounds,
- chemically diverse molecules.

Negative information is essential for improving the surrogate.

---

21. Retrain or Fine-Tune the Molecular Surrogate

Using the expanded dataset:

$$\mathcal{D}_{k+1},$$

update:

$$f_{\theta_k} \to f_{\theta_{k+1}}.$$

The scoring model should become progressively more accurate around chemically promising regions explored by the generative model.

Monitor:

- predictive error,
- calibration,
- uncertainty quality,
- out-of-distribution performance,
- ranking performance,
- correlation with true MD rescue scores.

---

22. Update the Generative Model

Use the improved surrogate model to provide better guidance for the molecular generator.

Depending on the generative architecture, this may involve:

- conditional generation,
- classifier guidance,
- reinforcement-learning-style optimization,
- score guidance,
- fine-tuning on high-performing molecules,

- rejection sampling,
- ranking generated molecules with the surrogate.

---

23. Repeat the Closed Active-Learning Loop

Repeat:

$$\boxed{\begin{aligned}&\text{Generate} \rightarrow \text{Predict} \rightarrow \text{Select} \rightarrow \text{MD}\\ &\rightarrow \text{Encode} \rightarrow \text{Score} \rightarrow \text{Expand dataset} \rightarrow \text{Retrain}\end{aligned}}$$

until:

- computational resources are exhausted,
- improvements plateau,
- sufficiently strong candidate molecules emerge,
- or candidates are ready for experimental validation.

Overall Architecture

The complete project contains three principal machine-learning systems.

Model 1 — Molecular Dynamics Representation Model

$$\boxed{\text{MD trajectory} \rightarrow z_{\text{trajectory}} \rightarrow R_{\text{WT rescue}}}$$

Purpose:

> Determine how closely the dynamics of a mutant or ligand-bound system resemble the wild-type dynamical state.

This model includes:

- frame encoder,
- temporal encoder,
- trajectory decoder,
- frame decoder,
- WT reference distribution,
- rescue scoring mechanism.

Model 2 — Molecular Rescue Surrogate

$$\boxed{\text{molecule} \rightarrow \hat{R}_{\text{WT rescue}}}$$

Purpose:

> Predict the MD-derived rescue score without running an expensive molecular dynamics simulation.

This model serves as a cheap approximation to the MD oracle.

Model 3 — Molecular Generator

$$\boxed{\text{generative model} \rightarrow \text{candidate rescue molecules}}$$

Purpose:

> Generate new compounds predicted to move Y220C dynamics toward the WT dynamical state.

The surrogate model provides guidance, while MD simulations provide the final high-fidelity evaluation.

Core Scientific Hypothesis

The central hypothesis of SAWNERGY becomes:

> **A learned trajectory-level representation of molecular dynamics can quantify restoration of mutant p53 dynamics toward the wild-type dynamical state, and this learned rescue signal can subsequently guide the iterative discovery of novel Y220C rescue compounds.**

The most important intermediate question is therefore:

> **Does distance from the learned WT trajectory distribution provide a reproducible and biologically meaningful measurement of Y220C rescue?**

The generative molecular-design loop should only be trusted after this question has been convincingly answered.

Final Closed-Loop System

$$
\begin{aligned}
\text{Molecule} &\rightarrow \text{Y220C MD simulation} \\
&\rightarrow \text{trajectory representation} \\
&\rightarrow \text{WT rescue score} \\
&\rightarrow \text{molecular surrogate training} \\
&\rightarrow \text{surrogate-guided generation} \\
&\rightarrow \text{new molecules} \\
&\rightarrow \text{MD simulation} \\
&\rightarrow \text{new rescue measurements} \\
&\rightarrow \text{dataset expansion} \\
&\rightarrow \text{model improvement} \\
&\rightarrow \cdots
\end{aligned}
$$

The resulting system is an iterative combination of:

representation learning + molecular dynamics + surrogate modeling + active learning + generative molecular design.